

BACKEND ENGINEERING · TAKE-HOME ASSESSMENT

Multi-Tenant Wallet API

Stack Django + DRF **Time** ~4–5 hours **Deliverable** GitHub repository

We'd like you to build a small wallet / ledger service where multiple tenants (merchants or organizations) operate on the same platform, but each tenant's data is fully isolated from the others. This mirrors the kind of system we work on, so it's a good way to see how you approach money handling and multi-tenancy.

The Task

Build a REST API (Django + DRF) that supports:

- Creating a tenant
- Creating a user / wallet under a tenant
- Deposit funds into a wallet
- Withdraw funds (reject if balance is insufficient)
- Transfer funds between two wallets *of the same tenant*
- Get a wallet's balance and a paginated transaction history

Multi-Tenancy Requirements

- Every request is scoped to a tenant, resolved from an API key or `X-Tenant-ID` header.
- A tenant must never be able to read or affect another tenant's wallets, users, or transactions.
- Transfers across different tenants must be rejected.

Wallet / Ledger Requirements

- Every balance change is recorded as an immutable transaction — the ledger is the source of truth, not just a mutable balance field.
- Transfers are atomic: both sides succeed or neither does.
- Deposit / withdraw / transfer endpoints are idempotent (accept a client-provided `idempotency_key` so a retried request never double-charges) — scoped per tenant.
- Money is stored as integer minor units (paise / cents) or `DecimalField` — never a float.
- Sensible validation and clear error responses.

What We'll Look At

- Correct DB transactions and row locking (`select_for_update`) to avoid race conditions.
- Clean tenant isolation — no way to leak or cross data.
- Ledger as source of truth; idempotency that actually works under retries.
- Data modeling, API design, and naming.
- Tests for core flows: insufficient funds, concurrent transfers, duplicate idempotency key, and cross-tenant access being blocked.

- A README with setup steps and any trade-offs or assumptions you made.

Tech

- Django + Django REST Framework
- Any database (PostgreSQL preferred)
- Docker is a plus, but optional

Scope & Submission

- Please aim for roughly 4–5 hours — we're not looking for production polish, just clean, correct, well-reasoned code.
- Share a GitHub repo (public or invite us), with a README explaining how to run it.

If anything is unclear, just ask — sensible assumptions are completely fine, as long as you note them in your README.